

ConstructoCompare PRO

Avance Final

Estudiantes: Samuel Bueno
Israel Gonzalez
Javiera Ramírez
Genara Alarcón
Docente: Marco Valenzuela
Fecha: 01-06-2026

Índice

1. Contexto del Proyecto y Modelo de Solución	5
1.1 Modelo de Negocio y Propuesta de valor	5
1.2 Definición del Mercado Objetivo	5
2. Descripción del Problema u Oportunidad	6
2.1 Análisis de Causas Raíz (Modelo Ishikawa)	6
Modelo Ishikawa	6
2.2 Análisis de Efectos e Impacto en el Entorno	7
2.3 Identificación de la Oportunidad	7
3. Objetivos del Proyecto	8
3.1 Objetivo General	8
3.2 Objetivos Específicos	8
3.3 Vinculación de Objetivos con la Problemática	8
4. Situación Inicial y Alcance	9
4.1 Situación Inicial	9
4.2 Alcance del proyecto	9
4.3 Entregables Técnicos	9
4.4 Supuestos y Restricciones	9
5. Planificación de actividades y responsabilidades	10
5.2 Cronograma Semestral (Carta Gantt)	11
Carta Gantt	12
6. Tecnologías y Lenguajes	12
6.1 Backend: FastAPI (Python)	12
6.2 Frontend: Next.js (React Framework)	13
6.3 Persistencia de Datos: PostgreSQL y Redis	13
6.4 Ingesta de Datos: Playwright	13
6.5 Arquitectura General del Sistema	13
7. Justificación de Tecnologías Cloud y Atributos de Calidad	14
7.1 Modelo de Servicio (Cloud Computing)	14
7.2 Atributos de Calidad y Viabilidad de la Solución	14
8. Metodología de Desarrollo y Ciclo de Vida del Software	14
8.1 Metodología Ágil: Scrum	15
8.2 Ciclo de Vida: Desarrollo Incremental e Iterativo	15
8.3 Atributos de Calidad y Viabilidad (Pilares Estratégicos)	15
9. Gestión de Calidad, Revisiones y Entregables	16
9.1 Herramientas de Gestión y Certificación de Avance	16
9.2 Estándares de Calidad y Entregas Parciales	16
10. Documentación, Estándares y Certificación de Calidad	16
10.1 Artefactos de Gestión y Trazabilidad	17
10.2 Estándares de Calidad y Validación Técnica	17
10.3 Certificación de Entrega Final	18
11. Diseño de la Solución y Lineamientos Técnicos	19
11.1. Arquitectura de Sistemas	19

A. Capa de Presentación (Frontend Service).....	19
B. Capa de Lógica de Negocio (Backend Service - Core API).....	20
C. Capa de Extracción e Integración (Scraper Workers)	20
D. Capa de Persistencia (Data Layer)	20
11.2. Especificación de Casos de Uso del Sistema	20
1. Definición de Actores	21
2. Procesos de Negocio Críticos (Clusters)	21
3. Reglas de Negocio Aplicadas.....	22
11.3. Modelo de Datos y Estrategia de Persistencia.....	22
Diagrama Entidad-Relación	22
12. Generación del Ambiente de Prueba y Configuración	24
12.1. Stack Tecnológico (Lenguajes y Runtimes).....	24
12.2. Bibliotecas y Dependencias Críticas.....	24
12. 3. Herramientas Necesarias	25
El siguiente procedimiento garantiza la replicabilidad del sistema en cualquier estación de trabajo:	25
13. Desarrollo e Implementación de la Solución	26
13.1. Implementación del Backend y Capa de Datos	26
13.2. Desarrollo de Motores de Ingesta.....	26
13.3. Interfaz de Usuario y Experiencia	26
13.4. Atributos de Seguridad del Producto	27
13.5 Procedimiento de Backup y Restauración mediante Branching.....	27
13.6. Configuración del Servidor Cloud (Ambiente de Pruebas).....	27
13.7. Evidencias de Configuración Exitosa.....	28
14. Aseguramiento de Calidad	29
14.1 Objetivo del QA	29
14.2 Estrategia de Pruebas	29
14.3 Gestión de Defectos	29
14.4 Definition of Done (DoD)	29
15. Plan de Pruebas	29
15.1 Objetivo del Plan de Pruebas	29
15.2 Alcance	30
15.3 Estrategia de Pruebas	30
16. Casos de Prueba	31
16.1 Matriz Resumen de Casos de Prueba	31
16.3 Cobertura de Pruebas	34
17. Aplicación de las Pruebas y Resultados	34
17.1 Ejecución de las Pruebas	34
17.2 Resultados Obtenidos	35
17.3 Hallazgos Detectados.....	35
17.4 Validación de Criterios de Aceptación	36
17.5 Conclusión de la Ejecución de Pruebas	36
18. Mejoras Implementadas a partir de las Pruebas	36
18.1 Mejorar en el Proceso de Extracción de Datos	36

18.2 Mejoras en la Calidad y Consistencia de los Datos	37
18.3 Mejoras en Rendimiento y Persistencia.....	37
18.4 Mejoras en la Interfaz de Usuario	37
18.5 Mejoras en el Proceso de Calidad	38
19. Evidencias de Validación y Configuración.....	38
19.1 Evidencias de Validación.....	38
19.2 Configuración del Ambiente y Pruebas.....	38
19.3 Configuración de Base de Datos	39
19.4 Herramientas de Gestión y Calidad	40
20. Conclusiones y lecciones aprendidas.....	40
20.1. Valor de Negocio Generado	40
20.2. Lecciones Aprendidas	41

1. Contexto del Proyecto y Modelo de Solución

ConstructoCompare Pro surge como una respuesta técnica a la ineficiencia estructural en la adquisición de suministros dentro de la industria de la construcción en Chile. Por lo que el proyecto se inserta en el mercado de la construcción en Chile, esto es un sector crítico ya que es afectado por la variabilidad económica y la opacidad de precios que obliga a los constructores a ejecutar procesos manuales de comparación.

1.1 Modelo de Negocio y Propuesta de valor

- Modelo de Negocio: Plataforma SaaS (Software as a Service) basada en la nube, diseñada para la escalabilidad y el acceso remoto desde cualquier dispositivo.
- Propuesta de Valor (Visión): Democratizar el acceso a precios en tiempo real que centraliza la oferta de los principales retailers, empoderando a profesionales para tomar decisiones financieras estratégicas, blindadas contra la inflación mediante un "Power Stack" de alto rendimiento (FastAPI, Next.js, Playwright). La solución permite que las Pymes compitan en igualdad de condiciones con grandes empresas al automatizar la comparación de valores en una sola herramienta.

1.2 Definición del Mercado Objetivo

El sistema impacta directamente en tres perfiles de usuarios con necesidades operativas distintas:

1. Pymes constructoras: Requieren optimizar presupuestos para licitaciones competitivas y control en su flujo de caja.
2. Maestros y Contratistas: Profesionales con alta movilidad que compran materiales a diario y requieren validación de precios en terreno.
3. Personas naturales enfocadas en la maximización del ahorro en proyectos de inversión personal.

2. Descripción del Problema u Oportunidad

El entorno de la construcción en Chile presenta una asimetría de información crítica que impide la eficiencia operativa de los actores del sector. Se identifica una problemática multidimensional donde el proceso de cotización actual es calificado como "arcaico", generando un impacto directo en la rentabilidad de los proyectos.

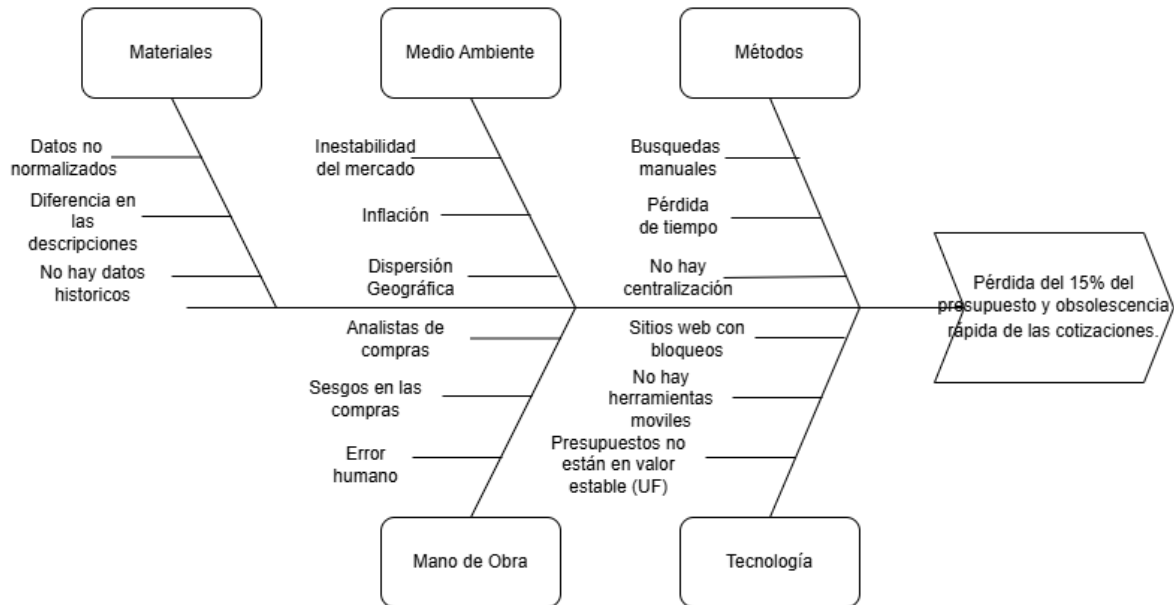
2.1 Análisis de Causas Raíz (Modelo Ishikawa)

La problemática se origina por la convergencia de factores estructurales y tecnológicos:

1. **Dispersión de Datos (Silos de Información):** Los grandes retailers (Sodimac, Easy e Imperial) operan con ecosistemas cerrados, obligando al usuario a una navegación fragmentada que consume entre 3 y 5 horas semanales.
2. **Ambigüedad Semántica y Falta de Normalización:** La inexistencia de un estándar de nomenclatura para materiales de construcción dificulta la comparación técnica inmediata entre proveedores.
3. **Volatilidad Macroeconómica:** La inestabilidad del peso chileno y la inflación constante provocan una variación de precios que el constructor no puede monitorear manualmente de forma eficiente.
4. **Brecha Tecnológica en Terreno:** La falta de herramientas móviles optimizadas para el cálculo presupuestario obliga a los contratistas a depender de métodos manuales o herramientas de texto poco precisas.

Modelo Ishikawa

Para profundizar en el origen de las ineficiencias detectadas, se ha elaborado un Diagrama de Ishikawa. Este análisis permite desglosar las causas técnicas, metodológicas y del entorno que derivan en la pérdida de rentabilidad del mercado objetivo.



Nota: La visualización del Modelo de Ishikawa se encuentra en el Anexo 1: Modelo Ishikawa ConstructoCompare Pro.

2.2 Análisis de Efectos e Impacto en el Entorno

La persistencia de estas causas genera efectos negativos que comprometen la viabilidad de las Pymes y proyectos personales:

1. Degradación del Capital de Trabajo: Se estima una pérdida de hasta un 15% del presupuesto debido a decisiones de compra basadas en información incompleta o desactualizada.
2. Obsolescencia Inmediata de Cotizaciones: Los presupuestos pierden validez en cuestión de días, lo que genera retrasos en el inicio de obras y renegociaciones constantes con los clientes finales.
3. Costo de Oportunidad Administrativo: El tiempo dedicado a la recolección manual de datos resta horas críticas que deberían enfocarse en la supervisión técnica y ejecución de la obra.
4. Desventaja Competitiva: Las Pymes constructoras, al no contar con departamentos de adquisiciones dedicados, quedan en una posición vulnerable frente a grandes empresas con mayores recursos tecnológicos.

2.3 Identificación de la Oportunidad

Ante este escenario, surge la oportunidad de implementar una solución de Inteligencia de Datos que centralice la oferta mediante un ID Maestro y blinde la información financiera a través de la conversión automática a UF (Unidad de Fomento), transformando un proceso reactivo en una ventaja estratégica competitiva.

3. Objetivos del Proyecto

3.1 Objetivo General

Digitalizar y optimizar la cotización de materiales mediante una plataforma centralizada que reduzca tiempos de búsqueda y mejore la precisión presupuestaria frente a la inflación

3.2 Objetivos Específicos

- Objetivo Específico 1: Implementar una arquitectura de microservicios resiliente con un motor de ingesta asíncrono (Python/Playwright) capaz de capturar precios de 3 retailers líderes del mercado nacional.
- Objetivo Específico 2: Diseñar un catálogo normalizado en PostgreSQL con lógica de Fuzzy Matching (ID Maestro) para garantizar que el usuario al comparar artículos técnicamente idénticos a pesar de las variaciones en la descripción por tiendas.
- Objetivo Específico 3: Integrar un módulo de sincronización automática con indicadores económicos oficiales (UF) para indexar las cotizaciones, asegurando su validez financiera frente a la inflación.
- Objetivo Específico 4: Desarrollar una interfaz progresiva (Next.js) optimizada para dispositivos móviles que permita la exportación de presupuestos en formato PDF, facilitando la toma de decisiones en terreno

3.3 Vinculación de Objetivos con la Problemática

La definición de estos objetivos responde directamente a las causas raíz analizadas previamente:

1. Respuesta a la Dispersión de Datos: El Objetivo Específico 1 soluciona la fragmentación de información mediante la automatización de la ingesta asíncrona.
2. Respuesta a la Ambigüedad Semántica: El Objetivo Específico 2 (ID Maestro) elimina la duplicidad y falta de normalización en el catálogo.
3. Respuesta a la Volatilidad Económica: El Objetivo Específico 3 provee la resiliencia financiera necesaria mediante la indexación automática a la UF.
4. Respuesta a la Brecha Tecnológica: El Objetivo Específico 4 entrega una herramienta profesional diseñada para el uso crítico en terreno.

4. Situación Inicial y Alcance

4.1 Situación Inicial

Actualmente, el proceso de cotización en el sector construcción es descentralizado y manual. Los usuarios deben consultar individualmente los sitios web de cada retail, comparar características técnicas sin un estándar común y registrar datos en herramientas no especializadas, lo que deriva en una alta tasa de error humano y pérdida de vigencia de los precios ante la inflación.

4.2 Alcance del proyecto

El alcance de **ConstructoCompare Pro** ha sido diseñado para responder directamente a los objetivos estratégicos del proyecto:

1. Gestión de Datos y Comparación: El sistema contempla la búsqueda centralizada y comparación multi-retailer mediante el uso de un ID Maestro para normalizar productos idénticos.
2. Inteligencia Presupuestaria: Incluye la conversión automática de listas de materiales a UF y el almacenamiento de históricos para análisis de tendencias.
3. Operatividad en Terreno: El sistema permite la gestión de listas de cotización personalizadas y la exportación de reportes técnicos en formato PDF para su uso inmediato en obra.

4.3 Entregables Técnicos

Para validar el cumplimiento del alcance, se definen los siguientes entregables:

1. Diseño: Diagrama de arquitectura de microservicios y Diccionario de Datos (DDL) en 3FN.
2. Producto: Prototipo funcional (MVP) desplegado en la nube.
3. Documentación: Manual de Usuario y Reporte de Aseguramiento de Calidad (QA) con pruebas unitarias.

4.4 Supuestos y Restricciones

Supuestos: Se asume la disponibilidad operativa de las APIs de indicadores económicos (mindicador.cl) y el acceso irrestricto a los catálogos públicos de los retailers seleccionados.

Restricciones Técnicas y Éticas: La extracción de datos se realizará bajo un estricto cumplimiento de las políticas definidas en el archivo robots.txt de cada retailer, respetando los tiempos de espera y las rutas permitidas para garantizar un scraping ético y no invasivo.

1. El sistema es estrictamente comparativo; no permite la compra directa (transacciones) dentro de la aplicación, delegando la adquisición al proveedor original mediante Deep Links.
2. La actualización de precios no es en tiempo real continuo, sino asíncrona (diaria), para evitar la saturación de los servidores de origen y prevenir bloqueos de IP.

5. Planificación de actividades y responsabilidades.

El proyecto se ejecuta bajo metodología Scrum, organizado en 5 sprints de dos semanas cada uno. El equipo distribuye las responsabilidades según sus áreas de especialización, manteniendo una colaboración continua durante todo el ciclo de desarrollo.

Javiera (Product Owner / Data Engineer): Responsable de la gestión y priorización del Product Backlog, diseño de la arquitectura de datos, persistencia de información, normalización de datos y despliegue de servicios.

Samuel (Scrum Master / Backend Developer): Responsable de la arquitectura backend, desarrollo de APIs, implementación de scrapers, lógica de comparación de precios y facilitación del proceso Scrum mediante la eliminación de impedimentos técnicos.

Israel (Frontend Developer / UX Designer): Responsable del diseño de interfaces, experiencia de usuario, integración frontend con servicios backend y desarrollo de funcionalidades visuales de comparación y gestión de cotizaciones.

Genara (QA Engineer / DevOps): Responsable de la validación de calidad del sistema, pruebas funcionales, control de integridad de datos, auditorías de seguridad y monitoreo del cumplimiento de los criterios de aceptación definidos para cada sprint.

5.1 Gestión del Backlog y Estimación

La priorización del Product Backlog se realizó considerando el valor de negocio, dependencias técnicas, riesgo de implementación y aporte al Producto Mínimo Viable (MVP). Las Historias de Usuario fueron organizadas en tres épicas principales: Gestión de Inventario Automatizada (Scraping), Motor de Comparación y Cálculo, y Panel de Usuario y Presupuestos.

Para la estimación del esfuerzo se utilizó la escala de Fibonacci (1, 2, 3, 5, 8 y 13 puntos), permitiendo reflejar la complejidad relativa y la incertidumbre técnica de cada Historia de Usuario. Las funcionalidades consideradas críticas para el núcleo del sistema, como el motor de extracción de datos, la arquitectura de comparación de productos y la gestión de cotizaciones, recibieron las estimaciones más altas debido a su impacto transversal en la solución.

5.2 Detalle del Product Backlog (Anexo 2)

La descomposición completa de los requerimientos se encuentra documentada en el Anexo 2: Product Backlog Priorizado. Este artefacto contiene las 20 Historias de Usuario del proyecto, organizadas por épicas y distribuidas en los cinco sprints definidos para la ejecución.

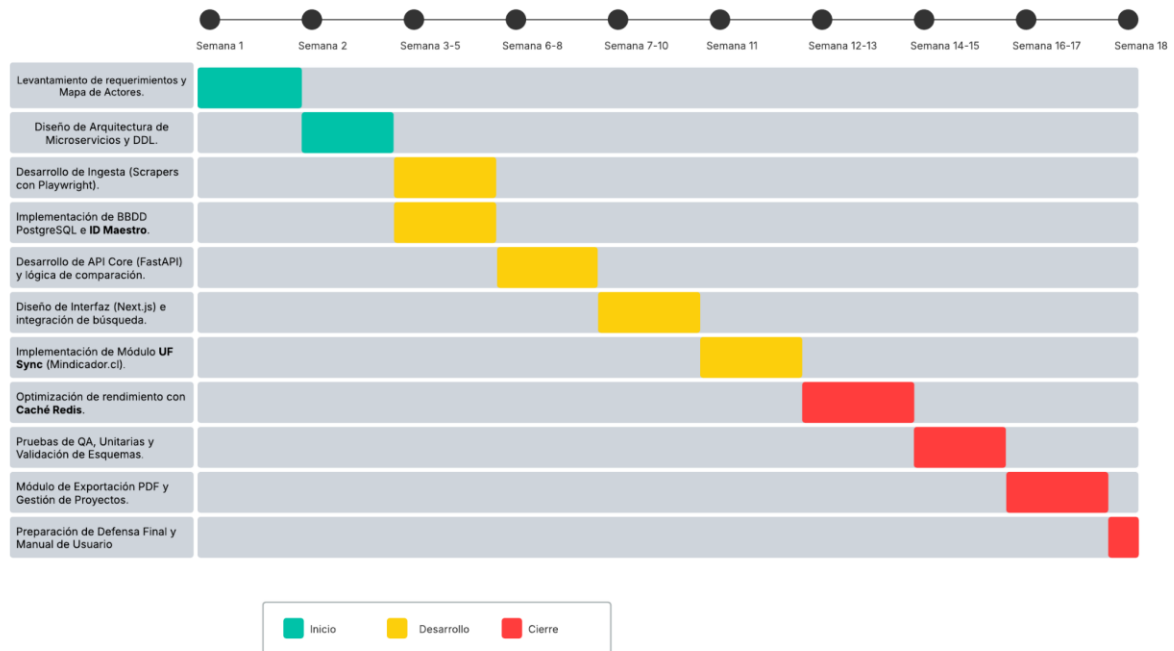
Para cada Historia de Usuario se especifican su descripción, criterios de aceptación, responsable, prioridad y estimación en puntos de historia. El Product Backlog constituye la principal fuente de planificación del proyecto y sirve como base para la elaboración de los Sprint Backlogs, el seguimiento de la velocidad del equipo y la validación de los incrementos entregados al finalizar cada sprint.

5.2 Cronograma Semestral (Carta Gantt)

Para garantizar el cumplimiento de los objetivos en las 18 semanas de plazo, se ha diseñado una Carta Gantt (elaborada en la herramienta de diagramación Lucidchart) que desglosa el proyecto en tres fases críticas: Inicio, Desarrollo y Cierre. Esta planificación permite una visualización clara de las dependencias técnicas y la asignación de responsabilidades.

1. Fase de Inicio (Semanas 1-2): Se centra en la definición de la arquitectura de microservicios y el diseño del modelo de datos. Es aquí donde se establecen las bases de la lógica del ID Maestro, asegurando que la estructura de la base de datos soporte la normalización desde el primer día.
2. Fase de Desarrollo (Semanas 3-11): Es la etapa de mayor carga operativa. Se trabaja en paralelo la ingesta de datos (Scrapers) y la construcción de la API. Se ha reservado un bloque significativo para la integración del módulo UF Sync, reconociendo su importancia crítica para la resiliencia financiera del sistema.
3. Fase de Cierre y Optimización (Semanas 12-18): A diferencia de modelos lineales, nuestro cierre incluye un periodo robusto de QA y optimización mediante Redis. Esto asegura que el producto final no solo sea funcional, sino también rápido y seguro bajo estándares profesionales.

Carta Gantt



Nota: La visualización de la planificación se encuentra en el Anexo 3: Carta Gantt ConstructoCompare Pro.

6. Tecnologías y Lenguajes.

La selección del stack tecnológico para ConstructoCompare Pro se basa en criterios de alto rendimiento, seguridad y mantenibilidad, siguiendo los estándares actuales de la industria para arquitecturas de microservicios .

6.1 Backend: FastAPI (Python)

Se ha seleccionado FastAPI como núcleo del servidor debido a su naturaleza asíncrona, permitiendo gestionar múltiples consultas a retailers de forma concurrente sin bloquear el hilo principal.

1. Validación con Pydantic: Garantiza la integridad de los datos mediante la validación de esquemas en tiempo real, asegurando que la información de precios capturada cumpla con el formato técnico requerido antes de su persistencia.
2. Documentación Automática: Implementa Swagger/OpenAPI de forma nativa, facilitando la integración con el frontend y reduciendo los tiempos de desarrollo.

6.2 Frontend: Next.js (React Framework)

Para la interfaz de usuario se utilizará Next.js, priorizando la velocidad y la adaptabilidad en dispositivos móviles.

1. Server-Side Rendering (SSR): Permite un renderizado rápido de las comparativas de precios, mejorando la experiencia del usuario en terreno con conexiones de red inestables.
2. Optimización Móvil: Facilita la creación de una interfaz progresiva que responde a la necesidad de los contratistas de visualizar presupuestos directamente en la obra.

El diseño visual de los formularios y pantallas principales se encuentra documentado en el Anexo 7: MockUps de Interfaz de Usuario, elaborados previo al desarrollo de cada sprint.

6.3 Persistencia de Datos: PostgreSQL y Redis

La arquitectura de datos utiliza un modelo híbrido para maximizar la consistencia y la velocidad:

1. PostgreSQL: Motor relacional robusto elegido para garantizar la integridad de las transacciones y la consistencia del ID Maestro bajo la Tercera Forma Normal (3FN).
2. Redis: Implementado como capa de caché para almacenar las consultas más frecuentes y los valores de la UF del día, logrando tiempos de respuesta inferiores a los 200ms

6.4 Ingesta de Datos: Playwright

Dada la complejidad de los sitios web de retail, se ha optado por Playwright para el motor de scraping asíncrono.

1. Evasión de Bloqueos: Capacidad superior para simular navegación humana, manejar carga dinámica de JavaScript y evadir sistemas básicos de detección de bots, asegurando la continuidad de la captura de precios.

6.5 Arquitectura General del Sistema

Para garantizar una solución escalable y de alta disponibilidad, se ha diseñado una arquitectura basada en servicios desacoplados que separa las responsabilidades de captura de datos, persistencia y lógica de negocio. Esta estructura permite que el sistema sea resiliente ante cambios en las plataformas de los retailers (Sodimac, Easy e Imperial) y asegure tiempos de respuesta óptimos para el usuario final.

Este diseño permite el procesamiento independiente de la información mediante un pipeline que asegura la calidad del dato antes de su visualización. El detalle técnico

de los componentes, el flujo de datos y el diagrama de arquitectura detallado se presentan en la sección 11 (Diseño de la Solución) de este informe, donde se establecen los lineamientos para el desarrollo eficiente de la plataforma.

7. Justificación de Tecnologías Cloud y Atributos de Calidad

La arquitectura de ConstructoCompare Pro se despliega bajo un modelo de computación en la nube para garantizar la escalabilidad y disponibilidad exigida por el sector construcción. La elección de servicios SaaS y PaaS responde a la necesidad de minimizar costos operativos (OpEx) y maximizar la entrega de valor al cliente .

7.1 Modelo de Servicio (Cloud Computing)

1. SaaS (Software as a Service): La entrega final al usuario es un SaaS accesible vía web, eliminando la necesidad de instalaciones locales por parte de las Pymes o contratistas, facilitando el acceso multiplataforma en terreno.
2. PaaS (Platform as a Service): Utilizaremos servicios como Render o Railway para el despliegue de la API y la base de datos, permitiendo escalabilidad automática sin gestionar servidores físicos (IaaS), reduciendo costos operativos.

7.2 Atributos de Calidad y Viabilidad de la Solución

Para asegurar que la herramienta sea una solución de grado profesional, se han documentado los siguientes atributos siguiendo las mejores prácticas de la industria:

1. Integridad y Precisión: El uso de PostgreSQL y la validación nativa con Pydantic aseguran que los datos de precios no se corrompa durante la ingesta. La lógica de ID Maestro garantiza que la comparación sea técnicamente precisa, vinculando productos idénticos de distintos retailers .
2. Confiabilidad: Al desplegar en PaaS, el sistema cuenta con respaldos automáticos y una disponibilidad del 99.9% (Uptime), asegurando que el constructor pueda consultar sus proyectos en cualquier momento crítico de la obra.
3. Oportunidad: La actualización asíncrona (diaria) y el uso de Redis garantizan que la información sea oportuna y se entregue con una latencia mínima (menor a 200ms), permitiendo la toma de decisiones rápidas frente a cambios de precios .
4. Seguridad: Se implementa cifrado de datos en tránsito (HTTPS) y una gestión robusta de identidad mediante Tokens, protegiendo la información de los proyectos y presupuestos de cada usuario.

8. Metodología de Desarrollo y Ciclo de Vida del Software

La naturaleza de ConstructoCompare Pro, caracterizada por la dependencia de datos externos y la volatilidad económica, exige un enfoque de desarrollo que combine agilidad con una arquitectura robusta .

8.1 Metodología Ágil: Scrum

Se ha seleccionado el marco de trabajo **Scrum** debido a que permite un desarrollo incremental y adaptativo, ideal para gestionar la incertidumbre propia del Web Scraping

1. Sprints Quincenales: Permiten entregar funcionalidades incrementales (MVPs) que son validadas rápidamente frente a cambios en la estructura web de los retailers.
2. Gestión del Backlog: La priorización basada en Fibonacci asegura que el equipo se enfoque primero en las tareas de mayor valor y riesgo técnico.

8.2 Ciclo de Vida: Desarrollo Incremental e Iterativo

El proyecto sigue un ciclo de vida incremental que responde a las dimensiones de los requerimientos y plazos definidos :

1. Planificación y Análisis: Definición de la arquitectura de microservicios y el ID Maestro para garantizar la integridad desde el origen .
2. Construcción (Sprints): Desarrollo paralelo de la ingesta (Samuel), la base de datos (Javiera) y la interfaz (Israel) .
3. Aseguramiento de Calidad (QA): Fase final de pruebas unitarias y de integración para validar la precisión de los datos y la seguridad de los tokens.

8.3 Atributos de Calidad y Viabilidad (Pilares Estratégicos)

Bajo las mejores prácticas de la industria, la viabilidad de la solución se sustenta en los siguientes atributos documentados :

1. Integridad (ID Maestro): Normalización de productos idénticos para evitar la duplicidad de información y garantizar que la comparación sea técnicamente coherente .
2. Confiabilidad y Resiliencia: El sistema implementa una estrategia de caché en Redis. Ante fallos en la carga dinámica de los retailers o de la API de indicadores, el sistema servirá los últimos datos válidos almacenados.
3. Precisión y Oportunidad: La indexación obligatoria a la UF vía API de mindicador.cl asegura que la información entregada responda con exactitud a los requerimientos financieros del cliente en un entorno inflacionario .
4. Seguridad: Gestión de acceso mediante sesiones persistentes y validación de esquemas con Pydantic para proteger la integridad de los proyectos guardados .

9. Gestión de Calidad, Revisiones y Entregables

La gestión del proyecto ConstructoCompare Pro se rige por estándares de la industria que garantizan la trazabilidad y la calidad técnica en cada fase del ciclo de vida del software .

9.1 Herramientas de Gestión y Certificación de Avance

Para garantizar la transparencia y el cumplimiento riguroso de los estándares de gestión de proyectos, el equipo utiliza un ecosistema de herramientas digitales que certifican el progreso parcial :

1. Gestión del Flujo de Trabajo (Jira): Se utiliza un tablero Jira para la visualización del Sprint Backlog. Cada Historia de Usuario transita por estados claros (Por hacer, En Proceso, Impedimentos, Testing, Terminado), permitiendo una trazabilidad total desde la HU1 hasta la HU20.
2. Sincronización Diaria (Daily Scrum): El equipo realiza reuniones diarias de 15 minutos para responder: ¿qué se hizo ayer?, ¿qué se hará hoy? y ¿existen impedimentos?. Esto asegura que las desviaciones en la Carta Gantt se detecten de forma inmediata.
3. Registro de Obstáculos (Impediment Log): Se mantiene un registro formal de impedimentos técnicos, tales como cambios inesperados en los selectores de los retailers o latencias en la API de Mindicador.cl . Este log permite documentar las soluciones aplicadas (como el uso de Redis o ajustes en Playwright) para mantener la continuidad del proyecto .

9.2 Estándares de Calidad y Entregas Parciales

La certificación del avance del producto de software se realiza bajo el estándar de "Definition of Done" (DoD), el cual exige:

1. Validación de Criterios de Aceptación (AC): Cada funcionalidad debe superar los tests definidos en el Anexo 2 .
2. Integración Técnica: El código debe estar integrado en el repositorio y validado mediante los esquemas de Pydantic.
3. Documentación Actualizada: Cada entrega parcial debe reflejarse en la actualización del Manual de Usuario y el reporte de avance de la solución tecnológica .

10. Documentación, Estándares y Certificación de Calidad

Para asegurar que ConstructoCompare Pro cumpla con los estándares de calidad y gestión de proyectos vigentes, se ha implementado un sistema de documentación rigurosa que garantiza la trazabilidad entre los requerimientos del cliente y los entregables técnicos.

10.1 Artefactos de Gestión y Trazabilidad

El proyecto se apoya en una estructura documental y de gestión que permite asegurar la trazabilidad completa de los requerimientos, el seguimiento del avance y la validación de los incrementos generados durante cada sprint.

1. Impact Mapping: Metodología utilizada para alinear los objetivos estratégicos del proyecto con las funcionalidades implementadas, asegurando que cada entregable contribuya directamente a la propuesta de valor de la plataforma.
2. Product Backlog Priorizado (Anexo 2): Registro oficial de las 20 Historias de Usuario organizadas en tres épicas (Gestión de Inventario Automatizada, Motor de Comparación y Cálculo, y Panel de Usuario y Presupuestos). Cada historia incorpora prioridad, estimación mediante escala de Fibonacci, criterios de aceptación y planificación por sprint.
3. Sprint Backlogs: Subconjuntos del Product Backlog definidos durante cada Sprint Planning, donde se establecen las Historias de Usuario comprometidas, responsables asignados y objetivos específicos de cada iteración.
4. Scrumboard: Herramienta visual utilizada para monitorear el estado de avance de las tareas, permitiendo identificar elementos pendientes, en desarrollo, en validación y completados.
5. Burndown Chart: Indicador de seguimiento empleado para controlar la evolución del trabajo pendiente durante cada sprint y verificar el cumplimiento de la capacidad comprometida por el equipo.
6. Daily Tracker e Impediment Log: Registro continuo de actividades, acuerdos y bloqueos técnicos detectados durante la ejecución del proyecto, facilitando la gestión temprana de riesgos y la toma de decisiones correctivas.
7. Sprint Review y Sprint Retrospective: Artefactos de inspección y mejora continua utilizados al cierre de cada sprint para validar los incrementos desarrollados, recopilar retroalimentación y definir acciones de optimización para las siguientes iteraciones.
8. Jira: Plataforma central de gestión utilizada para mantener la trazabilidad de las Historias de Usuario, tareas técnicas, incidencias, evidencias de desarrollo y seguimiento del avance del proyecto.

10.2 Estándares de Calidad y Validación Técnica

La calidad del producto se garantiza mediante un conjunto de actividades de validación ejecutadas durante cada sprint, permitiendo verificar el cumplimiento de los

criterios de aceptación definidos para las Historias de Usuario y asegurar la estabilidad de la solución.

1. Validación de Criterios de Aceptación:

Cada Historia de Usuario es validada mediante casos de prueba diseñados para comprobar el cumplimiento de sus criterios de aceptación, asegurando que la funcionalidad implementada entregue el valor esperado por el usuario.

2. Pruebas Funcionales e Integración:

Se ejecutan pruebas de humo, integración y validación de flujos completos entre frontend, backend y capa de datos, verificando la correcta interacción entre los distintos componentes de la plataforma.

3. Pruebas de Rendimiento y Concurrencia:

El sistema es sometido a pruebas de estrés, carga, volumen y concurrencia para evaluar su comportamiento ante múltiples usuarios y grandes volúmenes de información, garantizando estabilidad y escalabilidad operacional.

4. Control de Calidad de Datos:

Se implementan mecanismos de validación y normalización de datos durante el proceso de extracción y transformación, asegurando consistencia, integridad y confiabilidad de la información utilizada por la plataforma.

5. Control de Versiones y Trazabilidad:

El uso de un repositorio centralizado y una gestión basada en Scrum permite mantener un historial auditable de cambios, entregas e incidencias, facilitando el seguimiento de la evolución del producto durante todo el ciclo de desarrollo.

10.3 Certificación de Entrega Final

Al finalizar el ciclo de desarrollo, el equipo entregará la solución tecnológica acompañada de la documentación necesaria para su operación, mantenimiento y validación. La certificación de entrega considerará:

- Cumplimiento de las Historias de Usuario definidas en el Product Backlog.
- Validación funcional mediante los casos de prueba ejecutados durante los sprints.
- Evidencia de las Sprint Reviews y entregables incrementales desarrollados durante el proyecto.
- Manual de Usuario con las funcionalidades principales del sistema.
- Reporte de calidad y resultados de pruebas realizadas por el equipo.

- Repositorio de código fuente con control de versiones y trazabilidad de cambios.

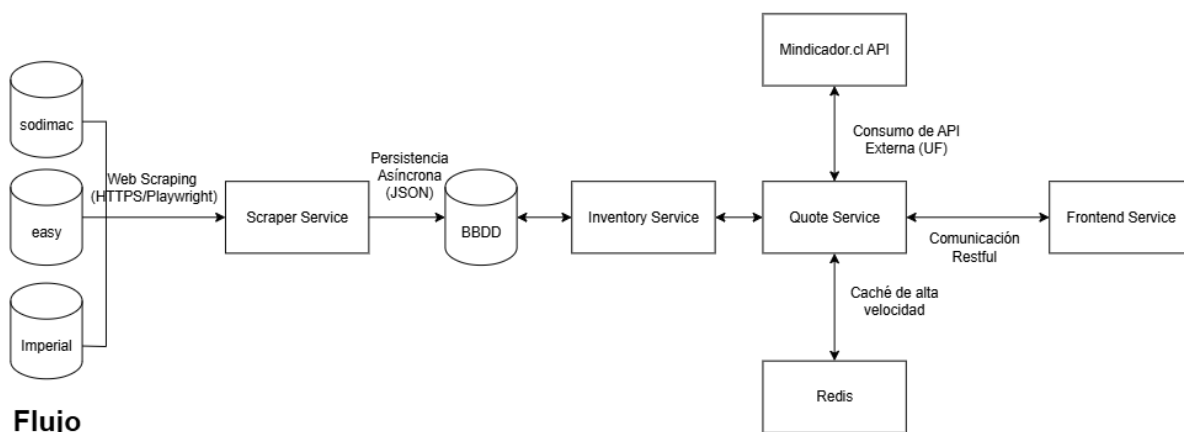
De esta manera se asegura que el producto entregado cumple los requisitos funcionales, de calidad y de gestión definidos durante el desarrollo, proporcionando una plataforma operativa para la comparación de precios y gestión de cotizaciones de materiales de construcción.

11. Diseño de la Solución y Lineamientos Técnicos

Esta sección detalla los documentos y diagramas técnicos que definen las características funcionales y estructurales de **ConstructoCompare PRO**, estableciendo la hoja de ruta para un desarrollo alineado con la metodología Scrum y las tecnologías de vanguardia seleccionadas.

11.1. Arquitectura de Sistemas

El diseño de **ConstructoCompare PRO** se estructura bajo un modelo de microservicios desacoplados. El siguiente diagrama de contenedores detalla las responsabilidades tecnológicas y los protocolos de comunicación definidos para la solución:



Nota: El diagrama detallado de la infraestructura, junto con la descripción exhaustiva del pipeline de datos (desde la ingesta con Playwright hasta la entrega en Next.js), se encuentra documentado en el Anexo 4: Arquitectura de Sistemas y Flujo de Datos.

A continuación, se detalla la responsabilidad de cada nodo crítico del sistema:

A. Capa de Presentación (Frontend Service)

- Tecnología: Next.js + Tailwind CSS.
- Responsabilidad: Proporcionar una interfaz de usuario (UI) responsiva y de alto rendimiento. Esta capa es responsable de la orquestación visual de los resultados comparativos y la gestión de la experiencia de usuario (UX).

- Interacción: Consume datos de forma asíncrona desde el Backend Service mediante protocolos REST, asegurando que la interfaz no se bloquee durante los procesos de búsqueda masiva.

B. Capa de Lógica de Negocio (Backend Service - Core API)

- Tecnología: FastAPI (Python 3.11+).
- Responsabilidad: Actúa como el cerebro del sistema. Gestiona la lógica del Motor de Comparación, la integración con indicadores financieros (API UF) y el "Firewall de Datos" mediante modelos de Pydantic.
- Interacción: Recibe las peticiones del Frontend, consulta la Capa de Persistencia y, en caso de ser necesario, dispara los procesos de actualización de precios.

C. Capa de Extracción e Integración (Scraper Workers)

- Tecnología: Playwright (Python).
- Responsabilidad: Ejecución de agentes automatizados que navegan y extraen información en tiempo real desde Sodimac, Easy e Imperial.
- Validación: Cada dato extraído es validado por el Backend antes de su procesamiento, asegurando que solo información íntegra (SKU, nombre y precio) ingrese al pipeline.

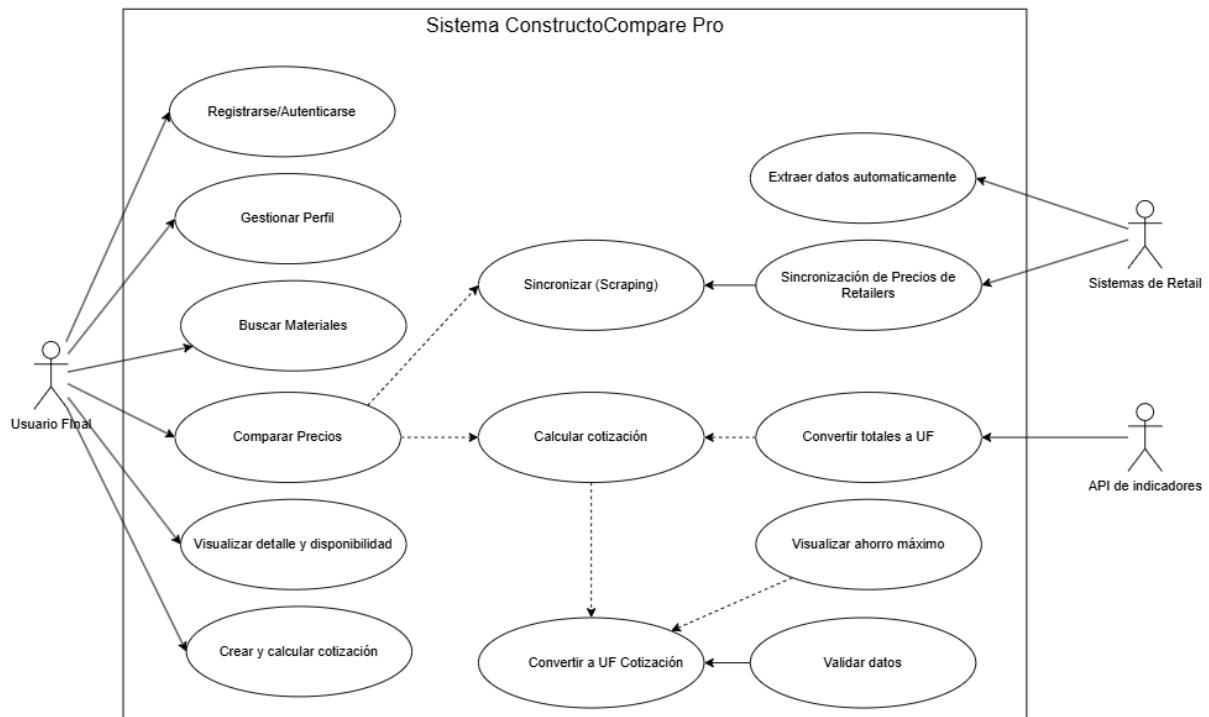
D. Capa de Persistencia (Data Layer)

- Tecnología: PostgreSQL 15.
- Responsabilidad: Almacenamiento del Catálogo Maestro (ID Maestro), histórico de precios y perfiles de usuario.
- Optimización: Implementa índices GIN y funciones de Fuzzy Matching (Trigramas) para resolver la problemática de la disparidad de nombres entre tiendas, permitiendo cruces de datos con una latencia inferior a 200ms.

11.2. Especificación de Casos de Uso del Sistema

Para garantizar que el desarrollo cumpla con las funcionalidades críticas identificadas en el Backlog, se ha modelado la interacción entre los distintos actores y el sistema. Este diagrama establece el alcance funcional y los límites de la solución ConstructoCompare PRO.

Diagrama General de Casos de Uso



Nota: La visualización del Diagrama general de caso de uso se encuentra en el Anexo 5: Diagrama General de Casos de Uso.

Descripción y Análisis de los Casos de Uso

El diagrama anterior modela el comportamiento del sistema ConstructoCompare PRO, definiendo los límites de la solución y las interacciones entre los actores y los procesos internos. A continuación, se detallan los lineamientos técnicos derivados de este diseño:

1. Definición de Actores

- **Usuario Final (Actor Principal):** Representa al constructor o jefe de obra que busca optimizar presupuestos. Tiene acceso a la gestión de perfil y a las funciones de búsqueda y cotización.
- **Sistemas de Retail (Actor Secundario):** Plataformas externas (Sodimac, Easy, Imperial) que actúan como fuentes de datos. Interactúan con el sistema de forma pasiva a través del motor de sincronización.
- **API de Indicadores Económicos (Actor Secundario):** Servicio externo que provee el valor de la UF en tiempo real, fundamental para la resiliencia financiera de las cotizaciones.

2. Procesos de Negocio Críticos (Clusters)

Para facilitar un desarrollo eficiente, los casos de uso se han agrupado en los siguientes flujos operativos:

- Flujo de Comparación y Cálculo (UC01, UC02, UC03): * La funcionalidad de Comparar Precios Multitienda incluye (<<include>>) obligatoriamente la búsqueda de materiales.
 - Este flujo establece que la respuesta del sistema debe presentarse en una matriz comparativa que destaque el "Ahorro Máximo" por ítem.
- Flujo de Ingesta y Calidad (UC05, UC07): * El caso de uso Sincronizar (Scraping) establece el lineamiento de automatización.
 - Este proceso incluye de forma obligatoria la Validación de Integridad de Datos (UC07). Aquí se implementa el uso de esquemas Pydantic, actuando como un "firewall de datos" que rechaza cualquier entrada malformada desde los retailers para proteger la base de datos PostgreSQL.
- Flujo de Gestión UF (UC04): * Establece que todo cálculo de cotización debe integrarse con el conversor de moneda. El lineamiento técnico define el uso de Redis para cachear este valor por 24 horas, optimizando el rendimiento y evitando bloqueos por exceso de peticiones a la API externa.

3. Reglas de Negocio Aplicadas

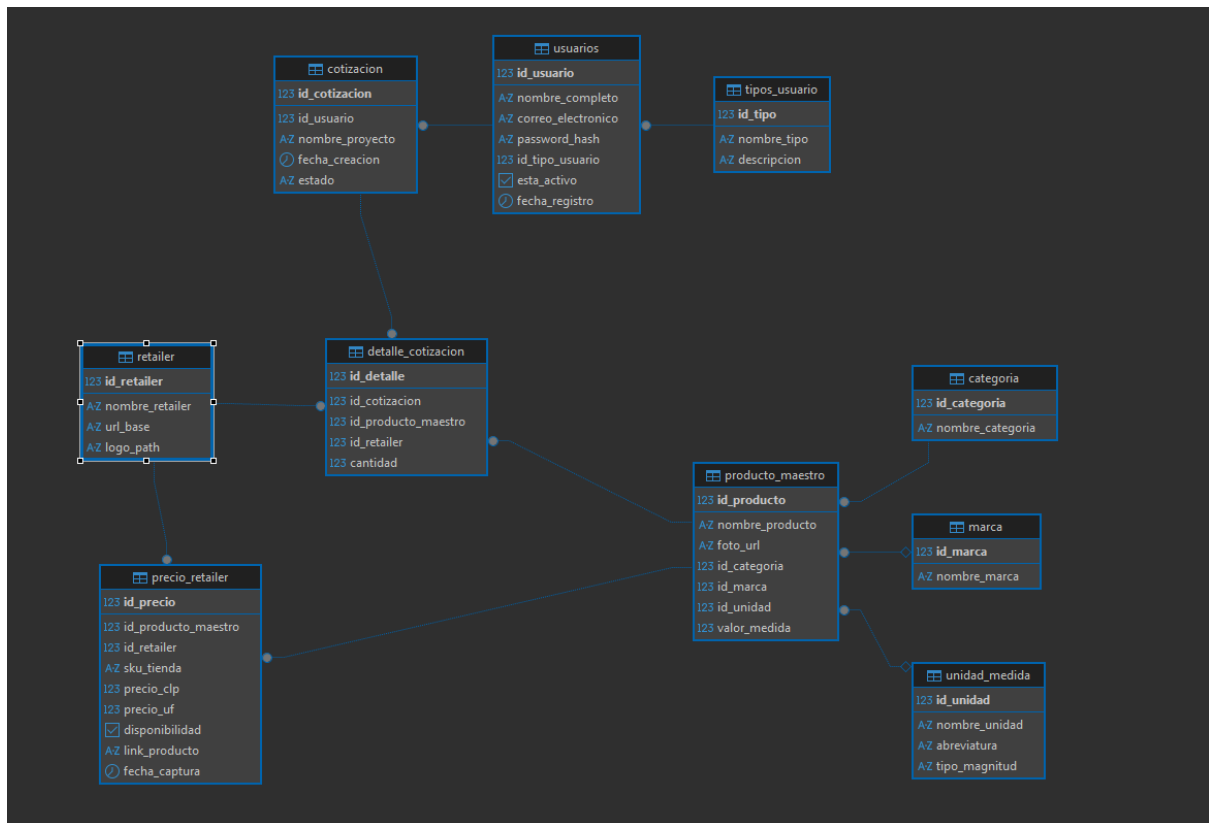
- Integridad Funcional: El sistema no permite generar una cotización (UC03) si el producto no ha pasado previamente por el filtro de validación (UC07).
- Seguridad: El acceso a la gestión de perfil (UC06) está condicionado a una autenticación exitosa mediante hashes de seguridad, garantizando la privacidad de los proyectos del usuario.

11.3. Modelo de Datos y Estrategia de Persistencia

El pilar fundamental de **ConstructoCompare PRO** es la integridad y normalización de la información ante la alta volatilidad del mercado de la construcción. El diseño del modelo de datos está orientado a resolver la disparidad de SKUs y nombres comerciales entre retailers mediante una estructura centralizada y optimizada.

Diagrama Entidad-Relación

El siguiente diagrama, diseñado bajo el estándar SQL relacional, establece la estructura lógica de la solución y las relaciones críticas para la comparación de precios:



Nota: La visualización del Diagrama de Entidad-Relación se encuentra en el Anexo 6: Diagrama Entidad-Relación.

Lineamientos Técnicos de Persistencia

Para garantizar un desarrollo eficiente y un producto final de calidad, se establecen los siguientes lineamientos de diseño:

1. Normalización mediante ID Maestro: La tabla PRODUCTO_MAESTRO actúa como el eje de la solución, asignando un identificador único a productos equivalentes de distintas tiendas (ej. Saco Cemento 25kg). Esto permite que el sistema realice cruces de datos precisos sin importar la nomenclatura interna de Sodimac, Easy o Imperial.
2. Precisión y Resiliencia Financiera:
 - Se utiliza el tipo de dato NUMERIC(12,5) para el campo PRECIO_UF tanto en las capturas de precios como en el detalle de las cotizaciones.
 - Esta precisión garantiza que los cálculos de presupuestos a largo plazo no pierdan exactitud por redondeo, permitiendo la visualización dinámica del valor en UF.
3. Regla de Negocio: Congelado de Precios: La entidad DETALLE_COTIZACION implementa un lineamiento crítico de calidad: al agregar un material a un presupuesto, se persiste el valor unitario (CLP y UF) del momento exacto. Esto protege al usuario final de las fluctuaciones de precios durante la fase de planificación de la obra.
4. Optimización y Rendimiento: La base de datos PostgreSQL 15 se configura con las siguientes características para cumplir el objetivo de latencia < 200ms:

- Índices GIN: Implementados para búsquedas rápidas en el catálogo maestro.
 - Fuzzy Matching (pg_trgm): Uso de la extensión de trigramas para resolver búsquedas de usuarios con errores tipográficos o nombres parciales.
5. Capa de Validación (Firewall de Datos): Antes de la persistencia en las tablas, todos los datos extraídos por los scrapers deben ser validados mediante esquemas de Pydantic. Esto asegura que registros con precios nulos o SKUs malformados sean rechazados en el origen, manteniendo la base de datos limpia e íntegra.
6. Seguridad y Auditoría: La tabla USUARIO establece el lineamiento de no almacenar contraseñas en texto plano, exigiendo el uso de hashes de seguridad (Bcrypt o Argon2). Además, se registra el ULTIMO_ACCESO para trazabilidad de la plataforma.

12. Generación del Ambiente de Prueba y Configuración

Esta sección describe los lineamientos para la creación de un ambiente de desarrollo y pruebas que replica fielmente las condiciones del entorno de producción. El objetivo es minimizar el riesgo de errores de despliegue y asegurar la estabilidad de ConstructoCompare PRO.

12.1. Stack Tecnológico (Lenguajes y Runtimes)

Para garantizar un alto rendimiento y escalabilidad, el sistema se basa en una arquitectura de microservicios desacoplados:

- Backend: Python 3.12+ (Motor de ejecución asíncrono).
- Frontend: Node.js 18.x o superior (Runtime para Next.js).
- Base de Datos: PostgreSQL 15+ (Relacional).
- Scrapers: Playwright con soporte para navegadores Chromium (Headless).

12.2. Bibliotecas y Dependencias Críticas.

El despliegue requiere la instalación de las siguientes bibliotecas base a través de los gestores de paquetes oficiales:

Backend (Python - pip install):

- FastAPI: Framework web de alto rendimiento.
- SQLAlchemy 2.0+: ORM para gestión de base de datos.
- asyncpg: Driver de conexión asíncrona nativa para PostgreSQL.
- Uvicorn: Servidor ASGI para producción.
- Pydantic v2: Validación de esquemas y tipos de datos.

Frontend (Node.js - npm/pnpm install):

- Next.js 14+: Framework de React para el lado del cliente y SSR.
- TypeScript: Tipado estático para la robustez del código.
- Vanilla CSS / CSS Modules: Estilización optimizada.

12. 3. Herramientas Necesarias

- Gestión de Procesos: Gunicorn con workers UvicornWorker para el backend.
- Proxy Inverso: Nginx para terminación SSL y balanceo de carga.
- Seguridad: Certificados Let's Encrypt (TLS 1.3).

12. 4. Instrucciones de Carga de Ambiente

El siguiente procedimiento garantiza la replicabilidad del sistema en cualquier estación de trabajo:

Backend

1. `python -m venv .venv`
2. `source .venv/bin/activate` # o `.\.venv\Scripts\activate` en Windows
3. `pip install -r requirements.txt`

Frontend

1. `cd frontend`
2. `pnpm install`
3. `pnpm dev`

12. 5. Gestión del Archivo de Configuración y Seguridad (.env)

Para el correcto funcionamiento del ecosistema, es obligatorio contar con un archivo `.env` en la raíz del proyecto (o en la carpeta backend/). Este archivo centraliza las credenciales sensibles y NO debe ser incluido en el control de versiones (Git).

Estructura Base del .env:

```
DATABASE_URL=postgresql+asyncpg://constructo_user:tu_password@localhost:5432/constructo_db
```

```
SQLALCHEMY_ECHO=false
```

```
AUTO_CREATE_TABLES=true
```

```
CORS_ORIGINS=http://localhost:3000,http://localhost:5173,http://127.0.0.1:3000,http://127.0.0.1:5173
```

```
JWT_SECRET_KEY={"tu_clave_secreta_para_jwt"}
```

```
JWT_ALGORITHM=HS256
```

```
JWT_ACCESS_TOKEN_EXPIRE_MINUTES=60
```

13. Desarrollo e Implementación de la Solución

En esta fase se materializa el diseño técnico en un producto funcional. El desarrollo de ConstructoCompare PRO se ha ejecutado bajo un enfoque de Clean Architecture, separando las responsabilidades para asegurar un código mantenible y escalable.

13.1. Implementación del Backend y Capa de Datos

El núcleo del sistema se desarrolló utilizando FastAPI, aprovechando su naturaleza asíncrona para gestionar múltiples peticiones simultáneas de comparación de precios sin bloqueos.

- **Acceso a Datos:** Se utilizó el ORM SQLAlchemy 2.0 junto con el driver asyncpg. Este lineamiento asegura transacciones atómicas y seguras, protegiendo la base de datos contra inyecciones SQL y optimizando el rendimiento de las consultas complejas de historial.

13.2. Desarrollo de Motores de Ingesta

Para obtener información en tiempo real, se desarrollaron microservicios de scraping basados en Playwright.

- **Extracción Resiliente:** Los scrapers navegan de forma automatizada por Sodimac, Easy e Imperial, transformando el HTML dinámico en estructuras JSON normalizadas.
- **Firewall de Datos (Calidad):** Siguiendo el compromiso de calidad, se integró Pydantic como capa de validación obligatoria. Cualquier dato extraído que presente inconsistencias (precios nulos o formatos de texto erróneos) es rechazado automáticamente en el origen, evitando que la "data basura" contamine los indicadores financieros del sistema.

13.3. Interfaz de Usuario y Experiencia

El frontend, desarrollado en Next.js 14, se diseñó bajo el lineamiento de "Mobile First" para facilitar el uso en terreno (obras).

- **Consumo Asíncrono:** La interfaz consume los endpoints del backend mediante Fetch API, permitiendo que la carga de precios multitienda sea fluida y no bloqueante.
- **Visualización UF:** Se implementó un componente de conversión dinámica que utiliza el valor capturado en el backend para mostrar presupuestos resilientes a la inflación, cumpliendo con uno de los requerimientos de mayor valor de negocio.

13.4. Atributos de Seguridad del Producto

Para garantizar un producto seguro:

1. Autenticación Robusta: La gestión de usuarios se implementa mediante el uso de tokens JWT (JSON Web Tokens) para el manejo de sesiones.
2. Protección de Credenciales: Las contraseñas nunca se almacenan en texto plano; se utiliza el algoritmo de hash Bcrypt, asegurando que incluso ante una eventual vulneración de la base de datos, la información sensible del usuario permanezca inaccesible.

13.5 Procedimiento de Backup y Restauración mediante Branching

En lugar de utilizar métodos tradicionales lentos (como `pg_dump`), el proyecto utiliza la tecnología de Branching de Neon DB. Este procedimiento permite crear una copia exacta de la base de datos de producción (incluyendo el MER completo y todos sus datos) de forma instantánea.

- Procedimiento de Backup: Se realiza un snapshot lógico de la rama principal (main) en un punto en el tiempo.
- Procedimiento de Restauración: Se crea una nueva rama denominada test-environment o qa-branch. Esta nueva rama hereda automáticamente toda la estructura (tablas, relaciones) y los datos de producción sin afectar la rama principal
- Ventaja Técnica: Esto garantiza que el ambiente de pruebas sea un reflejo 1:1 de producción, cumpliendo con el requisito de "verificación y validación" de forma inmediata y segura

13.6. Configuración del Servidor Cloud (Ambiente de Pruebas)

Para reflejar la configuración de producción en el entorno Cloud de pruebas, se han realizado los siguientes pasos:

1. Aislamiento de Conexión: Se configuró una nueva cadena de conexión (DATABASE_URL) específica para la rama de pruebas en el archivo `.env`.
2. Instalación de Dependencias: Se instalaron los lenguajes y bibliotecas necesarios (Python 3.13, FastAPI, SQLAlchemy) utilizando el gestor de paquetes pip y el entorno virtual `.venv`.
3. Sincronización de Datos: Gracias al branching, los 638 productos capturados por el scraper en producción están disponibles inmediatamente en el ambiente de pruebas para realizar auditorías de precios sin riesgo de corrupción de datos reales.

13.7. Evidencias de Configuración Exitosa

Branch	↑	Parent
 production Default		-
 Sprint 1 		production
 developer 		production

14. Aseguramiento de Calidad

14.1 Objetivo del QA

- Asegurar el cumplimiento de requisitos funcionales y no funcionales.
- Validar criterios de aceptación de las HU.
- Garantizar calidad, integridad y seguridad de la información.

14.2 Estrategia de Pruebas

- Pruebas funcionales.
- Pruebas de integración.
- Pruebas de rendimiento.
- Pruebas de seguridad.
- Pruebas de experiencia de usuario.

14.3 Gestión de Defectos

- Corrección y revalidación.
- Seguimiento mediante Impediment Log.

14.4 Definition of Done (DoD)

- Cumplimiento de criterios de aceptación.
- Pruebas aprobadas.
- Código integrado en repositorio.
- Validación del Product Owner.

15. Plan de Pruebas

15.1 Objetivo del Plan de Pruebas

El presente Plan de Pruebas tiene como objetivo verificar que el sistema Comparador Inteligente de Precios para el Rubro de la Construcción cumple correctamente con los requerimientos funcionales y no funcionales definidos durante el desarrollo del proyecto.

Las pruebas permiten validar la funcionalidad, calidad, rendimiento, seguridad e integridad de los distintos componentes del sistema, garantizando que las Historias de Usuario implementadas cumplen sus criterios de aceptación y que el producto se encuentra en condiciones de ser utilizado por los usuarios finales.

El proceso de validación considera pruebas sobre los módulos de Scraping, Motor de Comparación, Backend, Frontend y Gestión de Cotizaciones, utilizando distintos tipos de pruebas tales como:

- Pruebas de Humo (Smoke Testing).

- Pruebas Funcionales.
- Pruebas de Integración.
- Pruebas End-to-End (E2E).
- Pruebas de Estrés.
- Pruebas de Volumen.
- Pruebas de Regresión.
- Pruebas de Concurrencia.
- Pruebas de Seguridad y Calidad de Datos.

15.2 Alcance

El plan de pruebas contempla la validación de las funcionalidades desarrolladas durante los cinco sprints del proyecto:

- Gestión de usuarios y autenticación mediante JWT.
- Extracción automatizada de productos desde Sodimac, Easy e Imperial.
- Normalización y validación de datos.
- Motor de comparación de precios.
- Gestión de cotizaciones.
- Dashboard interactivo.
- Exportación de cotizaciones en PDF.
- Integridad y rendimiento de la plataforma.

15.3 Estrategia de Pruebas

La estrategia de validación se ejecutó utilizando un enfoque incremental alineado con la metodología Scrum.

Al finalizar cada Sprint se ejecutaron pruebas asociadas a las Historias de Usuario desarrolladas, permitiendo detectar tempranamente errores funcionales, problemas de rendimiento e inconsistencias de datos.

Las pruebas fueron diseñadas para verificar:

- Cumplimiento de criterios de aceptación.
- Integridad de la información procesada.
- Correcta comunicación entre componentes.
- Estabilidad bajo carga.
- Seguridad de acceso y protección de datos.
- Experiencia de usuario en la interfaz gráfica.

Los resultados obtenidos fueron documentados y utilizados como base para la implementación de mejoras continuas durante el desarrollo del proyecto.

16. Casos de Prueba

Los casos de prueba fueron diseñados para validar el cumplimiento de los criterios de aceptación definidos en las Historias de Usuario del Product Backlog. Cada caso de prueba busca verificar el comportamiento esperado del sistema ante distintos escenarios de uso, asegurando la correcta implementación de las funcionalidades desarrolladas durante los cinco sprints del proyecto.

Las pruebas fueron clasificadas según el tipo de validación requerida, incluyendo pruebas funcionales, de integración, de rendimiento, seguridad y experiencia de usuario.

El detalle completo de los procedimientos, datos de entrada, resultados esperados y evidencias obtenidas se encuentra disponible en el Anexo_8 _Plan_Pruebas_Funcionales_ConstructoCompare_Pro.

16.1 Matriz Resumen de Casos de Prueba

A continuación, se detalla la matriz maestra de control de calidad del proyecto. Esta matriz refleja con honestidad de ingeniería el estado real del MVP al cierre del ciclo actual, manteniendo los casos del Sprint 1 al 5 en estado Aprobado (Ejecutados).

ID	Componente / Módulo	Historia de Usuario (HU)	Objetivo de la Prueba	Criterio de Aceptación (DoD)	Estado
TC-S1-BE-01	Backend (FastAPI)	HU-J1: Persistencia de Usuarios	Verificar disponibilidad del servidor y bloqueo de registros anomalías (Smoke Test).	GET raíz responde HTTP 200. Bloqueo de correos duplicados mediante overrides de dependencia en < 0.85s.	Aprobado
TC-S1-BE-02	Backend (FastAPI)	HU-J1: Persistencia de Usuarios	Evaluar la resiliencia del endpoint /login ante picos masivos de tráfico concurrente.	100 usuarios simultáneos responden 401/200 con 0% de errores 500 en el servidor.	Aprobado
TC-S1-FE-01	Frontend (Next.js 14)	HU-I1 / HU-G1 / HU-S1	Validar el flujo completo automatizado del cliente (Login → Navegación → Búsqueda).	Acceso demo exitoso, inyección de sesión, render adaptativo de UF y búsqueda de "cemento" con Playwright.	Aprobado
TC-S1-SC-01	Scraping (Sodimac)	HU-S1: Extracción Base	Comprobar conexión y selectores funcionales del scraper core de Sodimac.	Navegación por sitemap.xml, parseo CSS correcto de metadatos esenciales y respeto a robots.txt.	Aprobado
TC-S2-FE-01	Frontend (Next.js 14)	HU-I2.1: Visualización Real	Validar la carga síncrona de múltiples retailers y el comportamiento de lazy loading.	Búsqueda multitienda en < 5s sin bloqueos visuales. Despliegue fluido al hacer scroll down.	Aprobado
TC-S2-SC-01	Scraping (Silver)	HU-S2.1: Motor Asíncrono	Validar la resiliencia y orquestación simultánea de Sodimac, Easy e	Extracción paralela estable con 15 workers concurrentes y reintentos	Aprobado

ID	Componente / Módulo	Historia de Usuario (HU)	Objetivo de la Prueba	Criterio de Aceptación (DoD)	Estado
			Imperial Scrapers.	automáticos ante categorías vacías.	
TC-S2-SC-03	Scraping (Silver)	HU-J2.1: Sanitización Precios	Verificar la transformación y limpieza de strings monetarios complejos en la Capa Silver.	Regex elimina caracteres extraños; casteo exitoso a int/float numérico no negativo y sin ceros.	Aprobado
TC-S2-SC-04	Scraping (Silver)	HU-G2.1: QA Ingesta y Logs	Auditar la consistencia estructural del dato y cumplimiento de esquemas antes de la carga.	Validación estricta de presencia de SKU, Nombre y URL de origen; 100% de cobertura multitienda en BD.	Aprobado
TC-S3-BE-01	Backend (PostgreSQL)	HU-G3.1: Tuning SQL	Evaluar la persistencia relacional y el pool de conexiones bajo consultas concurrentes masivas.	50 peticiones simultáneas distribuidas con asyncio.gather sobre /search con 0% de fallas 500.	Aprobado
TC-S3-BE-02	Backend (FastAPI)	HU-J3.1: ID Maestro Canónico	Validar la lógica de agrupamiento y eliminación de ofertas duplicadas del mismo retail.	Aserciones de unicidad unifican inventarios dispersos bajo un maestro común libre de redundancias.	Aprobado
TC-S3-FE-01	Frontend (Next.js 14)	HU-I3.1: Vista Side-by-Side	Certificar la reactividad del Badge "Mejor Precio" y la conmutación de divisas CLP/UF.	El cliente renderiza badges analíticos de costos y recalcula precios instantáneamente al activar el toggle UF.	Aprobado

ID	Componente / Módulo	Historia de Usuario (HU)	Objetivo de la Prueba	Criterio de Aceptación (DoD)	Estado
TC-S3-SC-01	Scraping (Gold)	HU-S3.1: Lógica is_cheapest	Validar el pipeline de datos Medallion completo (Bronze → Silver → Gold).	El motor Gold mapea URLs físicas y agrupa variantes usando tokens con un threshold_confident=50.	Aprobado
TC-S4-BE-01	Backend (FastAPI)	HU-S4.1 / HU-J4.1	Verificar el ciclo de vida completo transaccional del microservicio de cotizaciones.	Flujo CRUD E2E exitoso: POST (JSON complejo) → GET Historial → PUT (Cantidades) → Soft Delete.	Aprobado
TC-S4-FE-01	Frontend (Next.js 14)	HU-I4.1: Dashboard Cotizaciones	Evaluar la reactividad del estado del cliente ante modificaciones masivas de presupuestos.	Clics iterativos e hiper-rápidos en controladores +/- recalculan subtotales en 0.33s sin congelar el render.	Aprobado
TC-J5.1-01	Backend (Cloud)	HU-J5.1: Perfil de Usuario	Validar persistencia de perfiles y seguridad JWT avanzada en el entorno de producción cloud.	Actualización de metadatos de usuario protegidos por políticas RBAC con token firmado.	Aprobado
TC-S5.1-01	Scraping (Servicio)	HU-S5.1: Export PDF	Comprobar la generación asíncrona del reporte binario de la cotización y su descarga limpia.	Conversión de estructura JSON a flujo de bytes PDF legible y con diseño alineado.	Aprobado
TC-G5.1-01	Scraping (Módulo)	HU-G5.1: QA Seguridad Final	Ejecutar auditoría final de evasión de bloqueos de scrapers y	Reporte consolidado de logs sin filtraciones de credenciales ni	Aprobado

ID	Componente / Módulo	Historia de Usuario (HU)	Objetivo de la Prueba	Criterio de Aceptación (DoD)	Estado
			consolidación de telemetría de red.	bloqueos IP permanentes.	
TC-I5.1-01	Frontend (Next.js 14)	HU-I5.1: Dashboard Final	Certificar la renderización adaptativa de gráficos de series de tiempo e históricos de precios.	Carga de componentes analíticos basados en librerías de gráficos sin degradación de velocidad.	Aprobado

16.3 Cobertura de Pruebas

La ejecución de los casos de prueba permitió validar la totalidad de las funcionalidades comprometidas en el Product Backlog. La cobertura alcanzada considera las tres épicas definidas para el proyecto:

- E1 – Gestión de Inventario Automatizada (Scraping): validación de extracción, normalización y calidad de datos.
- E2 – Motor de Comparación y Cálculo: validación de agrupación de productos, comparación de precios y optimización de consultas.
- E3 – Panel de Usuario y Presupuestos: validación de autenticación, gestión de cotizaciones, exportación PDF y visualización de información.

En total se ejecutaron pruebas funcionales, de integración, rendimiento y seguridad, obteniendo resultados satisfactorios y permitiendo identificar oportunidades de mejora durante el proceso de desarrollo.

17. Aplicación de las Pruebas y Resultados

17.1 Ejecución de las Pruebas

Una vez definidos los casos de prueba y sus respectivos criterios de aceptación, se procedió a la ejecución de las pruebas funcionales sobre los módulos desarrollados durante los distintos sprints del proyecto.

Las pruebas fueron ejecutadas en un entorno controlado utilizando la arquitectura implementada para el sistema ConstructoCompare, considerando los componentes Backend (FastAPI), Frontend (Next.js), Base de Datos PostgreSQL y los módulos de extracción automatizada desarrollados con Playwright.

La validación se realizó utilizando datos reales obtenidos desde los retailers integrados en la plataforma (Sodimac, Easy e Imperial), permitiendo verificar el comportamiento del sistema bajo condiciones similares a las de operación productiva.

Las pruebas abarcaron las tres épicas definidas en el Product Backlog:

- Gestión de Inventario Automatizada (Scraping).
- Motor de Comparación y Cálculo.
- Panel de Usuario y Presupuestos.

17.2 Resultados Obtenidos

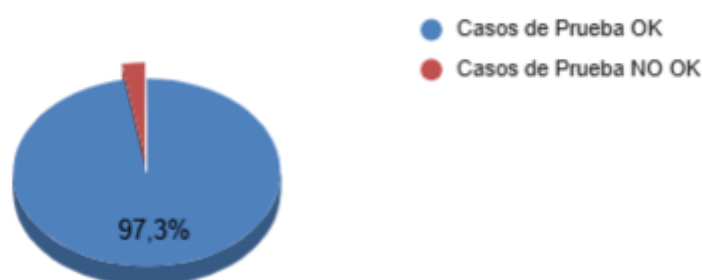
La ejecución de los casos de prueba permitió validar el cumplimiento de los criterios de aceptación definidos para las Historias de Usuario priorizadas del proyecto.

Resumen de resultados:

Estado	Cantidad
Casos Ejecutados	37
Casos Aprobados	1
Casos Rechazados	0
Casos Bloqueados	0
Porcentaje de Éxito	97,3%

Los resultados obtenidos demuestran que las funcionalidades implementadas cumplen con los requerimientos funcionales establecidos durante la planificación del proyecto.

Distribución por Casos de Prueba



Nota: La visualización del Gráfico y detalle se encuentra en el Anexo 9: Planilla Casos de Prueba

17.3 Hallazgos Detectados

Durante la ejecución de las pruebas se identificaron algunas situaciones que requirieron ajustes antes de considerar las funcionalidades como aprobadas:

- Diferencias en formatos de precios entre distintos retailers.
- Variaciones en la estructura HTML de algunas páginas utilizadas para scraping.
- Inconsistencias menores en la normalización de unidades de medida.
- Ajustes de rendimiento en consultas de comparación de productos.
- Optimización de carga de imágenes y visualización de resultados en la interfaz.

Todos los hallazgos fueron corregidos durante los sprints correspondientes y posteriormente validados mediante nuevas ejecuciones de prueba.

17.4 Validación de Criterios de Aceptación

La totalidad de los criterios de aceptación definidos para las Historias de Usuario fueron verificados mediante casos de prueba específicos.

Entre los aspectos validados destacan:

- Registro e inicio de sesión seguro de usuarios.
- Extracción automatizada de información desde múltiples retailers.
- Normalización de datos y precios.
- Comparación correcta de productos equivalentes.
- Identificación automática del menor precio disponible.
- Gestión de cotizaciones y presupuestos.
- Exportación de información en formato PDF.
- Visualización de gráficos históricos y paneles interactivos.
- Integridad y consistencia de los datos almacenados.

La evidencia obtenida confirma que el producto cumple con los objetivos funcionales definidos en el alcance del proyecto.

17.5 Conclusión de la Ejecución de Pruebas

La aplicación de las pruebas permitió verificar la estabilidad, confiabilidad y correcto funcionamiento de los componentes desarrollados durante el proyecto.

Los resultados obtenidos evidencian que la solución implementada satisface los requerimientos establecidos en el Product Backlog y se encuentra preparada para su utilización por parte de los usuarios finales, manteniendo niveles adecuados de calidad, integridad de datos y experiencia de usuario.

18. Mejoras Implementadas a partir de las Pruebas

La ejecución del Plan de Pruebas permitió identificar oportunidades de mejora tanto en la calidad de los datos como en la experiencia de usuario y el rendimiento general del sistema. A partir de los resultados obtenidos durante las validaciones funcionales, se implementaron diversas mejoras correctivas y preventivas que fortalecieron la estabilidad y confiabilidad de la plataforma.

18.1 Mejorar en el Proceso de Extracción de Datos

Durante las pruebas de scraping se detectaron diferencias en la estructura HTML de los distintos retailers, especialmente en Imperial y Easy. Para mitigar este problema se realizaron las siguientes mejoras:

- Implementación de una arquitectura modular basada en BaseStoreScraper, permitiendo adaptar cada retailer de manera independiente.
- Incorporación de mecanismos de espera dinámica y navegación automática para manejar contenido cargado mediante JavaScript.
- Normalización de URLs de productos para evitar registros duplicados.
- Optimización de selectores y validaciones para aumentar la estabilidad ante cambios menores en las páginas de origen.

18.2 Mejoras en la Calidad y Consistencia de los Datos

Las pruebas de validación evidenciaron la necesidad de unificar formatos provenientes de distintas fuentes. Como resultado se implementó:

- Normalización automática de precios mediante expresiones regulares.
- Conversión de valores numéricos a formatos estandarizados para cálculos posteriores.
- Homologación de categorías, marcas y unidades de medida entre retailers.
- Incorporación de validaciones mediante esquemas Pydantic para rechazar registros incompletos o inconsistentes antes de su almacenamiento.

18.3 Mejoras en Rendimiento y Persistencia

A partir de las pruebas de carga y procesamiento de datos se identificaron oportunidades de optimización:

- Implementación de índices en la base de datos para mejorar los tiempos de búsqueda y consulta.
- Optimización de consultas SQL utilizadas en los procesos de comparación.
- Escritura segura de archivos mediante mecanismos de persistencia atómica.
- Reducción de tiempos de procesamiento en la carga de grandes volúmenes de productos.

18.4 Mejoras en la Interfaz de Usuario

Las pruebas funcionales sobre la aplicación web permitieron realizar ajustes orientados a mejorar la experiencia del usuario:

- Integración de Lazy Loading para la carga progresiva de imágenes.
- Incorporación de Skeleton Loaders para representar estados de carga.
- Optimización del diseño responsivo para distintos tamaños de pantalla.
- Mejora en la visualización de resultados y tiempos de respuesta percibidos por el usuario.

18.5 Mejoras en el Proceso de Calidad

Como resultado de la incorporación de actividades formales de aseguramiento de calidad, se fortalecieron los mecanismos de control del proyecto mediante:

- Definición de casos de prueba asociados a criterios de aceptación.
- Validación sistemática de historias de usuario antes de su cierre.
- Registro y seguimiento de incidencias detectadas durante las pruebas.
- Incorporación de una etapa de revisión de calidad previa a la liberación de cada incremento de software.

Estas mejoras permitieron incrementar la confiabilidad de la plataforma, reducir errores durante la ejecución y asegurar que las funcionalidades desarrolladas cumplieran con los criterios de aceptación definidos en el Product Backlog.

19. Evidencias de Validación y Configuración

19.1 Evidencias de Validación

La validación del sistema se realizó mediante la ejecución de los casos de prueba definidos en el Plan de Pruebas. Las evidencias obtenidas permitieron verificar el cumplimiento de los criterios de aceptación asociados a las Historias de Usuario implementadas durante el desarrollo del proyecto.

Las evidencias recopiladas incluyen:

- Capturas de pantalla de la interfaz de usuario mostrando la ejecución exitosa de las funcionalidades.
- Resultados de autenticación de usuarios mediante registro e inicio de sesión.
- Evidencias de extracción y visualización de productos provenientes de los retailers integrados.
- Resultados de comparación de precios entre distintas tiendas.
- Validación de creación, edición y almacenamiento de cotizaciones.
- Evidencias de generación y descarga de documentos PDF.
- Registros de ejecución de pruebas funcionales y de integración.

- Logs de validación de datos y control de errores generados por el sistema.

Todas las evidencias fueron almacenadas como respaldo de la ejecución de pruebas y utilizadas durante la validación final del producto.

19.2 Configuración del Ambiente y Pruebas

Las pruebas fueron ejecutadas en un entorno controlado que replica las condiciones operativas definidas para el sistema.

Backend

- Python 3.12
- FastAPI
- SQLAlchemy
- PostgreSQL
- JWT Authentication
- Pydantic

Frontend

- Next.js
- React
- TypeScript
- Tailwind CSS

Extracción de Datos

- Playwright
- Arquitectura BaseStoreScraper
- Integración con retailers Sodimac, Easy e Imperial

Infraestructura

- Sistema Operativo Windows 11
- Docker
- Git y GitHub para control de versiones
- Jira para gestión y trazabilidad del proyecto

19.3 Configuración de Base de Datos

La validación se realizó utilizando una base de datos PostgreSQL con información obtenida desde los procesos de extracción y normalización implementados durante los sprints.

La base de datos de pruebas incluyó:

- Usuarios registrados.
- Productos provenientes de Sodimac.
- Productos provenientes de Easy.
- Productos provenientes de Imperial.
- Catálogo maestro de productos equivalentes.
- Cotizaciones generadas por usuarios.
- Historial de precios para validación de consultas.

Los datos utilizados permitieron verificar los procesos de búsqueda, comparación, almacenamiento y generación de reportes implementados en la solución.

19.4 Herramientas de Gestión y Calidad

Durante el proceso de validación se utilizaron las siguientes herramientas:

Herramienta	Propósito
Jira	Gestión de backlog, sprints y seguimiento de tareas
GitHub	Control de versiones
PostgreSQL	Persistencia de datos
Playwright	Extracción automatizada de información
FastAPI	Exposición de servicios backend
Next.js	Interfaz de usuario
Pydantic	Validación de esquemas y calidad de datos

La utilización de estas herramientas permitió mantener la trazabilidad completa entre requerimientos, historias de usuario, casos de prueba y resultados obtenidos durante la ejecución del proyecto.

20. Conclusiones y lecciones aprendidas

20.1. Valor de Negocio Generado

El desarrollo de ConstructoCompare PRO permitió construir una plataforma orientada a resolver una problemática real del sector de la construcción: la dificultad de comparar precios de materiales entre distintos proveedores de forma rápida, confiable y actualizada.

A lo largo de los cinco sprints definidos en el Product Backlog, el equipo logró implementar una solución funcional capaz de extraer información desde múltiples retailers, normalizar los datos obtenidos, comparar productos equivalentes y generar cotizaciones centralizadas para apoyar la toma de decisiones de compra.

La plataforma integra información proveniente de Sodimac, Easy e Imperial, permitiendo visualizar alternativas de compra en una única interfaz, reduciendo significativamente el tiempo requerido para realizar comparaciones manuales.

Los principales beneficios entregados por la solución son:

- Centralización de información proveniente de múltiples proveedores.
- Comparación automática de precios entre tiendas.
- Optimización del proceso de elaboración de cotizaciones.
- Gestión y almacenamiento de presupuestos por usuario.
- Generación de reportes y exportación de cotizaciones en formato PDF.
- Mayor trazabilidad y control sobre los costos asociados a proyectos de construcción.

Desde una perspectiva de negocio, la solución aporta valor tanto a pequeñas constructoras como a contratistas independientes y usuarios particulares, facilitando la toma de decisiones basadas en información actualizada y reduciendo el esfuerzo operativo asociado a la búsqueda de materiales.

20.2. Lecciones Aprendidas

Durante el desarrollo del proyecto se identificaron diversos aprendizajes técnicos y de gestión que contribuyeron al éxito de la implementación.

Uno de los principales aprendizajes fue la importancia de diseñar una arquitectura modular desde las primeras etapas del proyecto. La utilización de componentes desacoplados permitió incorporar nuevas funcionalidades y nuevos retailers sin afectar significativamente los módulos ya desarrollados, facilitando la escalabilidad de la solución.

Asimismo, la aplicación de la metodología Scrum permitió mantener una planificación ordenada del trabajo mediante la utilización de Product Backlog, Sprint Backlogs, Daily Meetings, Reviews y Retrospectives. Esto facilitó la detección temprana de riesgos y la adaptación continua frente a cambios técnicos y funcionales surgidos durante el desarrollo.

Otra lección relevante fue la incorporación temprana de actividades de aseguramiento de calidad. La ejecución sistemática de casos de prueba, junto con la validación de criterios de aceptación en cada sprint, permitió detectar errores antes de llegar a producción y aumentar la confiabilidad del producto final.

Finalmente, el proyecto evidenció la importancia de la colaboración multidisciplinaria dentro del equipo. La distribución clara de responsabilidades entre desarrollo backend, frontend, gestión de datos y aseguramiento de calidad permitió mantener una carga de trabajo equilibrada y asegurar el cumplimiento de los objetivos comprometidos en cada sprint.

Como resultado, el equipo logró entregar un producto funcional alineado con los requerimientos definidos inicialmente, validado mediante pruebas formales y respaldado por una documentación completa de gestión, desarrollo y calidad.